

RAPPORT DU PROJET DE BDD2

CHIABERGE Christopher,

ROULEAU Gwendoline,

DIALLO Mamoudou

01/02/2025

685L

INTRODUCTION.....	2
GESTION DU PROJET.....	2
TABLE ET TUPLES.....	3
DÉPENDANCES FONCTIONNELLES.....	4
Dépendances fonctionnelles non gardées pour une cohérence avec la réalité :.....	4
Modèle relationnelle de la table ReseauTan.....	4
PROCESSUS DE NORMALISATION.....	5
Calcul des clés avec la méthode LMR :.....	5
Calcul de la couverture minimale CV(DF) :.....	6
Normalisation par décomposition :.....	8
MODÈLE RELATIONNEL NORMALISÉ.....	10
COMPARAISON DES PERFORMANCES DES REQUÊTES.....	11
VU DU PROJET.....	13
Difficultés rencontrées :.....	13
Améliorations possibles :.....	13
CONCLUSION.....	14
RÉFÉRENCES.....	15

INTRODUCTION

Ce projet de base de données 2 a pour objectif le développement et l'amélioration d'une base de données.

Nous avons d'abord cherché un thème qui nous parle pour nous permettre d'être réaliste et avons trouvé plusieurs idées comme la représentation d'un refuge animalier, une plateforme de vente en ligne, une clinique médicale ou encore le réseau de transports de Nantes.

Nous avons choisi par la suite de représenter le réseau de transport de Nantes. C'est un modèle existant, et observable dans la vie courante, cela a pu nous aider à créer une représentation plus rapidement.

Nous allons créer une table avec tous les attributs que nous voulons avoir, définir les dépendances fonctionnelles puis la normaliser avec l'algorithme par décomposition puis l'algorithme de Bernstein.

Nous avons besoin de normaliser les tables dans les bases de données afin d'éviter les redondances, d'améliorer la cohérence et l'intégrité des données, de faciliter la maintenance et les mises à jour, d'améliorer la performance des requêtes et de faciliter l'évolution de la base de données.

GESTION DU PROJET

Le diagramme de Gantt demandé sera ici utilisé comme marqueur temporel. Nous avons décidé de ne pas attribuer de tâche personnelle pour permettre aux étudiants du groupe de pouvoir

expérimenter chaque tâche du projet mais avons rajouté un référent pour chaque grande étapes du projet .

DIAGRAMME DE GANTT BDD 2

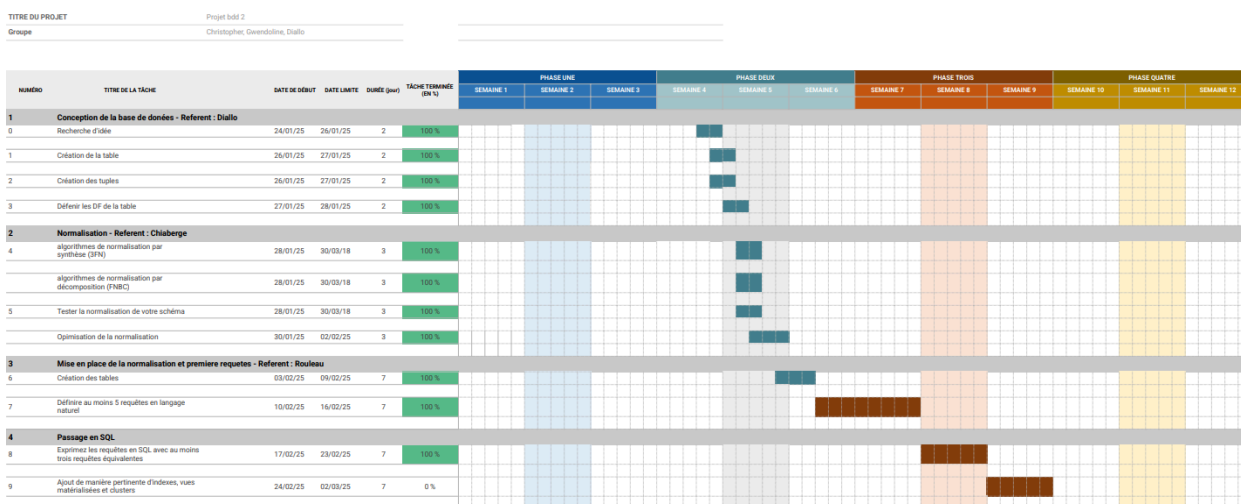


TABLE ET TUPLES

ReseauTan								
ligne_id	ligne_nom	ligne_type	arret_id	arret_nom	vehicule_id	vehicule_type	capacité	carburant
1	Ligne 1	Rail	1	Commerce	101	Tramway	250	Électrique
1	Ligne 1	Rail	3	Hôtel Dieu	102	Tramway	250	Électrique
2	Ligne 2	Rail	2	Beaujoire	102	Tramway	250	Électrique
3	C5	Route	3	Hôtel Dieu	201	Bus	50	Gaz_Naturel
4	N1	Fluviales	4	Gare Maritime	301	Navibus	95	Diesel
4	N1	Fluviales	5	Trentemoult	301	Navibus	95	Diesel
5	N3	Fluviales	6	Gare Sud	302	Navibus	95	Hydrogène

ReseauTan								
6	Ligne 4	Route	7	Hôtel de Région	401	BusWay	150	Électrique
7	N2	Fluviales	8	Île de Nantes	303	Navibus	95	Électrique
8	dédié	Route	NULL	NULL	501	ProxiTan	4	Diesel
9	12	Route	9	Université	202	Bus	50	Gaz_Naturel
10	Ligne 5	Route	10	Hangar à Banane	402	BusWay	150	Électrique

DÉPENDANCES FONCTIONNELLES

ligne_id → ligne_nom, ligne_type; (PC -> NC)

arret_id → arret_nom; (PC -> NC)

ligne_id, arret_id → arret_nom; (PC -> NC)

capacite → vehicule_type; (NC -> NC)

ligne_nom → ligne_id, ligne_type; (PC -> NC)

ligne_id, arret_id, vehicule_id → vehicule_type, capacite, carburant; (C -> NC)

ligne_id, vehicule_id → vehicule_type, capacite, carburant; (PC -> NC)

arret_id, vehicule_id → vehicule_type, capacite, carburant; (PC -> NC)

vehicule_id → vehicule_type, capacite, carburant; (PC -> NC)

Dépendances fonctionnelles non gardées pour une cohérence avec la réalité :

arret_nom → arret_id (en réalité il y a plusieurs arrêts qui ont le même nom donc nous avons de ne pas la garder pour être le plus proche possible de la réalité)

arret_id → ligne_type (ne représente pas correctement la réalité car un arrêt peut servir à des

bus ou à des trams donc nous avons de ne pas la garder pour être le plus proche possible de la réalité)

Modèle relationnelle de la table ReseauTan

ReseauTan
<u>ligne_id</u>
<u>arret_id</u>
<u>vehicule_id</u>
ligne_type
arret_nom
vehicule_type
capacite
carburant

Montrons que la table initiale n'est pas normalisé : (nous allons utiliser les clés calculées dans l'étape 1 du processus de normalisation)

Rappelons que nous considérons qu'une table est normalisée si elle est en forme 3FN ou en FNCB.

En observant les annotations **bleu** à la suite de chacune des dépendances fonctionnelles, nous sommes en 1 FN car les attributs sont atomiques, ce n'est pas en 2FN car nous avons plusieurs DF qui sont en (PC -> NC) donc vu que ça ne valide pas 2 FN, on n'est pas non plus en 3 FN et en FNBC.

PROCESSUS DE NORMALISATION

Pour faciliter la suite du projet nous avons décidé de renommer nos attributs ainsi :

ligne_id = A / ligne_nom = B / ligne_type = C / arret_id = D / arret_nom = E

vehicule_id = F / vehicule_type = G / capacite = H / carburant = I

Calcul des clés avec la méthode LMR :

L	M	R
F	A	C

D	B	E
	H	I
		G

En se référant aux dépendances fonctionnelles, nous observons que F^+ , D^+ et FD^+ ne sont pas suffisants pour atteindre l'univers.

Il faut donc utiliser les attributs du milieu. FDH^+ ne nous donne pas l'univers, seulement FDA^+ et FDB^+ peuvent atteindre l'univers.

F^+	D^+	FD^+	FDH^+	FDA^+	FDB^+
FGHI	DE	FGHIDE	FGHIDEH	FGHIDEABC	FGHIDEBAC

Notre modèle possède donc 2 clés : **key<FDA>** ou **key<FDB>**. Le choix de la clé pourrait avoir un impact sur l'optimisation de notre base et de nos algorithmes de normalisation.

Calcul de la couverture minimale CV(DF) :

Rappel des DF :

$A \rightarrow B, C$ $H \rightarrow G$ $D, F \rightarrow G, H, I$
 $B \rightarrow A, C$ $F \rightarrow G, H, I$ $A, F \rightarrow G, H, I$
 $D \rightarrow E$ $A, D \rightarrow E$ $A, D, F \rightarrow G, H, I$

En utilisant le split sur A et B nous obtenons :

$A \rightarrow B / A \rightarrow C / B \rightarrow A / B \rightarrow C$

La fermeture de F nous donne GHIF nous pouvons donc supprimer les attributs gauche des df comprenant à droite GHI, ce qui nous donne 4 fois $F \rightarrow GHI$, donc par redondance nous pouvons les supprimer.

Puis par split sur $F \rightarrow GHI$ nous obtenons :

$F \rightarrow I / F \rightarrow G / F \rightarrow H$

Nous observons que D^+ est suffisant pour obtenir E, donc la df $A, D \rightarrow E$ peut être supprimée par

redondance.

Par transitivité nous pouvons supprimer $A \rightarrow C$ et $F \rightarrow G$.

Aucune autre simplification ne peut être apportée nous avons alors la couverture minimale :

CV(DF) = $\{A \rightarrow B, B \rightarrow A, B \rightarrow C, D \rightarrow E, H \rightarrow G, F \rightarrow H, F \rightarrow I\}$

Normalisation par synthèse (Bernstein) :

Rappel de l'algorithme :

L'objectif de cet algorithme est de normaliser la table en 3FN tout en préservant les dépendances fonctionnelles (**spdf**) et en étant sans perte d'informations (**spi**). Nous devons d'abord déterminer les clés (ce que nous avons déjà fait) et calculer la couverture minimale de notre système de dépendances fonctionnelles, ensuite nous devons construire une relation par dépendances fonctionnelles. Ensuite on regroupe les relations qui ont la même partie gauche. On regroupe ensuite les relations qui ont leur ensemble d'attribut inclus dans un autre ensemble d'attribut. Une fois tous les regroupements possibles effectués, on regarde si une des relations contient une des clés trouvées précédemment, si ce n'est pas le cas on crée une dernière relation dans laquelle on ajoute la clé sans y mettre de dépendances fonctionnelles.

Avec **key<FDB>** ou **key<FDA>** et

CV(DF) = $\{A \rightarrow B, B \rightarrow A, B \rightarrow C, D \rightarrow E, H \rightarrow G, F \rightarrow H, F \rightarrow I\}$

Etape 1)

R1(AB) R2(BA) R3(BC) R4(DE) R5(HG) R6(EH) R7(FI)

Etape 2)

Regroupement des relations ayant même partie gauche :

R1(AB) R2,3(BAC) R4(DE) R5(HG) R6,7,8(EHI)

Etape 3)

R1 est compris dans R2 donc peut être supprimé :

R2,3(BAC) R4(DE) R5(HG) R6,7,8(EHI)

Etape 4)

Renommage des relations et création de R5(key) car R3(HG) ne contient pas de clé :

R1(BAC) R2(DE) R3(HG) R4(EHI) R5(FDA) en choisissant **key<FDA>**

R1(BAC) R2(DE) R3(HG) R4(EHI) R5(FDB) en choisissant **key<FDB>**

Test de la normalisation avec l'outil donner :

Supply a list of known functional dependencies:

```
ligne_id-->ligne_nom,ligne_type;
arret_id-->arret_nom;
vehicule_id-->vehicule_type,capacite,carburant;
ligne_id,arret_id-->arret_nom;
ligne_id,vehicule_id-->vehicule_type,capacite,carburant;
capacite-->vehicule_type;
ligne_id,vehicule_id,arret_id-->vehicule_type,capacite,carburant;
arret_id,vehicule_id-->vehicule_type,capacite,carburant;
ligne_nom-->ligne_id,ligne_type;
```

Functional dependencies are written as a list of attributes (the determiners), followed by an arrow (-->), followed by another list of attributes (the dependents). Each attribute should be separated by commas, and each dependency should be separated by a semi-colon, as in the example.

☒ 1. Find the Closure ☐ 2. Find a Key ☐ 3. Find the Minimal Cover ☐ 4. Decompose into 3NF Schemas ☐ 5. Find All

Submit

The closure is {ligne_id,arret_id,vehicule_id,ligne_nom,ligne_type,arret_nom,vehicule_type,capacite,carburant}

One possible key is {arret_id,vehicule_id,ligne_nom}

The minimal cover is { ligne_id-->ligne_nom; arret_id-->arret_nom; capacite-->vehicule_type; vehicule_id-->capacite; vehicule_id-->carburant; ligne_nom-->ligne_id; ligne_nom-->ligne_type }

We can decompose the single relation like so:

- Relation 1 is vehicule_id,capacite,carburant
- Relation 2 is ligne_nom,ligne_id,ligne_type
- Relation 3 is arret_id,vehicule_id,ligne_nom
- Relation 4 is arret_id,arret_nom
- Relation 5 is capacite,vehicule_type

Normalisation par décomposition :

Rappel de l'algorithme :

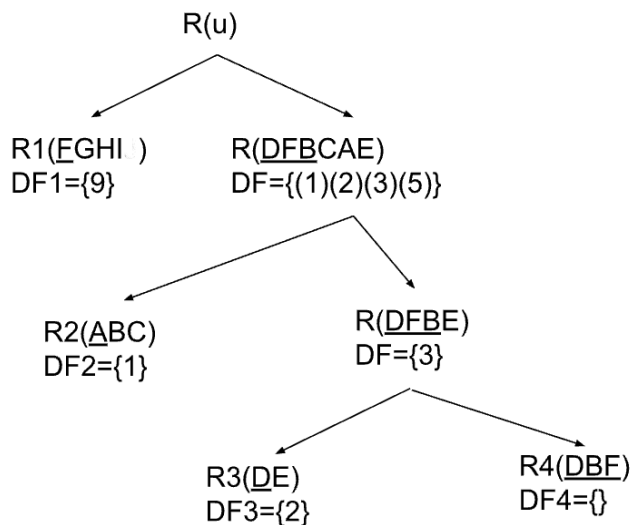
L'objectif de cet algorithme est de normaliser la table en FNBC en éliminant les dépendances

fonctionnelles indésirables et en conservant l'intégrité des données (**spi**). Dans cet algorithme nous sortons une à une les dépendances non triviales qui violent le principe de Boyce-Codd, grâce à cela on décompose petit à petit les relations pour créer des tables qui répondent aux critères de FNBC. Ce processus crée un arbre de décomposition dont on a juste à récupérer les feuilles pour avoir le nombre de tables à créer et leurs attributs. Il faut faire attention dans notre décomposition de toujours garder la clé choisie au début de l'algorithme dans le R présent à droite de l'arbre pour obtenir dans la dernière feuille, tous les attributs qui composent la clé.

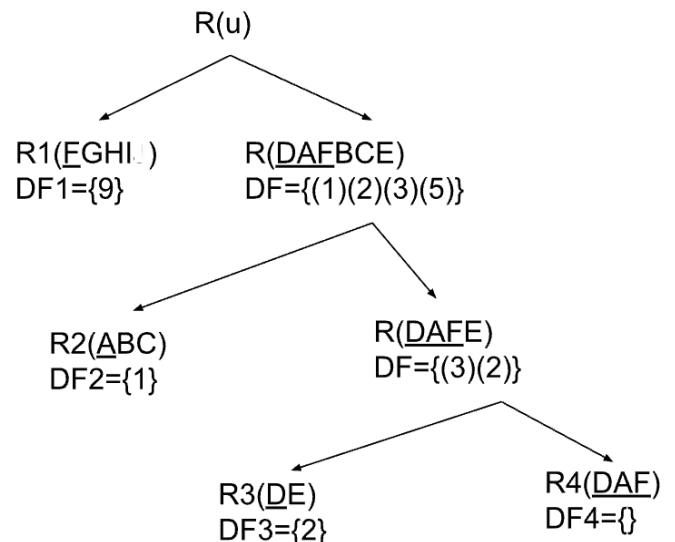
Rappel des DF :

- | | |
|--------------------------|-----------------------------------|
| (1) $A \rightarrow B, C$ | (6) $A, D, F \rightarrow G, H, I$ |
| (2) $D \rightarrow E$ | (7) $A, F \rightarrow G, H, I$ |
| (3) $A, D \rightarrow E$ | (8) $D, F \rightarrow G, H, I$ |
| (4) $H \rightarrow G$ | (9) $F \rightarrow G, H, I$ |
| (5) $B \rightarrow A, C$ | |

Avec **key<DFB>** :



Avec **key<DFA>** :



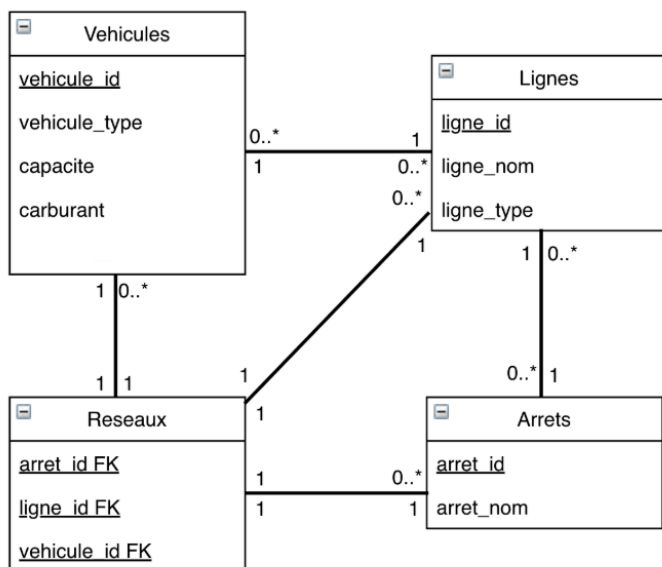
MODÈLE RELATIONNEL NORMALISÉ

Nous obtenons donc 4 modèles relationnels différents, 2 par algorithme de normalisation.

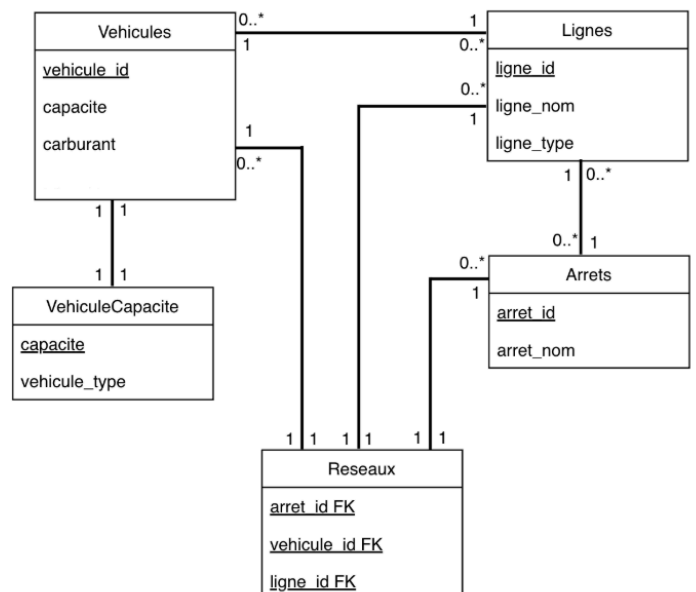
Le seul changement est le choix des clés dans la table réseaux, qui est soit *arret_id*, *ligne_id* et *vehicule_id* pour le choix de la clé ADF ou *arret_id*, *ligne_nom*, *vehicule_id* si on choisit la clé BDF, hors il est plus pertinent pour nous d'utiliser la clé AFD.

Nous obtenons donc 2 modèles relationnels différents :

Normalisation décomposition clé choisie : ADF



Normalisation Bernstein clé choisie : ADF



Le choix du modèle relationnel normalisé sera fait en fonction des requêtes les plus utilisées afin d'optimiser la base de données.

Nous avons décidé d'utiliser par la suite la normalisation obtenue avec le processus de décomposition car la version avec berstein nous serait utile si nous devions modifier régulièrement la capacité ou le type de véhicule sur les ligne, ce qui n'est pas le cas dans notre cas d'application, une capacité ne changera jamais pour un type de véhicule et les lignes de transport ne sont pas conçu pour permettre un changement de type de véhicule, il n'est donc pas utile de garder cette 5ème table.

COMPARAISON DES PERFORMANCES DES REQUÊTES

Nous avons choisie de réaliser 5 requêtes sur notre base de données :

- **1** : Une requête pour récupérer tous les véhicules ayant une capacité de plus de 100 et qui passe par l'arrêt Commerce
- **2** : Une requête pour trouver tous les véhicules électriques et leur capacité
- **3** : Une requête pour compter le nombre de véhicules par type d'énergie
- **4** : Une requête pour lister toutes les lignes avec leurs arrêts et véhicules associés
- **5** : Une requête pour trouver les arrêts desservis par des véhicules fluviaux (Navibus)

On aura 3 versions SQL pour chaque requête, nous pourrons ainsi étudier la rapidité d'exécution des 3 versions et décider laquelle choisir.

Requête 1 :

Version 1 :

version 2 :

Version 3 :

Requête 2 :

Version 1 :

version 2 :

Version 3 :

Requête 3 :

Version 1 :

version 2 :

Version 3 :

Requête 4 :

Version 1 :

version 2 :

Version 3 :

Requête 5 :

Version 1 :

version 2 :

Version 3 :

VU DU PROJET

Difficultés rencontrées :

Nous avons dû à plusieurs reprises revoir nos dépendances fonctionnelles et en avons trouvé après des premières tentatives de normalisation, ce qui les a rendues caduques. La couverture minimale a été aussi un peu dure à calculer, plusieurs règles n'étaient pas appliquées ce qui ne la rendait pas minimale.

Nous nous sommes rendu compte d'une erreur sur l'atomicité d'un de nos attributs, il s'agissait de l'attribut "accessibilité" qui comportait plusieurs valeurs possibles comme "Wifi","ascenseur",...

Hors cela ne satisfait pas la première forme normale et donc notre base de données n'était pas correctement normalisée. Nous avons fait le choix de supprimer cet attribut mais nous aurions aussi pu, pour chaque valeur qu'il prenait, créer un attribut de type booléen.

Améliorations possibles :

Nous avons une liste d'attributs qui étoufferait la base de données :

- Kilométrage du véhicule
- Année de mise en service du véhicule
- Heure de service de la ligne
- Nombre de véhicules sur la ligne
- Tranche de passage à un arrêt
- Les conducteurs / employés (nom,prénom,id,métier...)
- Marque véhicule (change les vitesses et capacités)
- Vitesse max et moyenne du véhicule
- Mode nocturne qui permet un arrêt à la demande

CONCLUSION

...

RÉFÉRENCES